

系统开发工具基础

肖扬

计算机科学与技术

学号：25020007140

2026 年 9 月 14 日

目录

1	实验环境说明	1
2	课堂学习部分	2
2.1	第 9 题从源码构建并在干净环境安装 Wheel	2
2.2	第 10 题让编程智能体进入可验证的修复循环	5
2.3	第 11 题把“无法处理”的协作材料改成可执行信息	10
2.4	第 12 题修复一个可复现的线性回归训练循环	13
3	课后习题	16
3.1	第 1 题创建并安装带有 pyproject.toml 的 Python 包	16
3.2	第 2 题使用现有的 pyproject.toml 项目执行构建并检查发布文件 . . .	19
3.3	第 3 题安装第二题构建的 Wheel 包并检查安装结果	22
3.4	第 4 题查看 Python 包的依赖信息和安装位置	23
3.5	第 5 题使用 Claude 智能体读取代码并自动生成说明文件	24
3.6	第 6 题让 Claude 智能体发现并修复 Python 程序中的错误	28
3.7	第 7 题编写一个最简单的 README	32
3.8	第 8 题编写一个规范的 Bug 报告	33
3.9	第 9 题手动规范 Python 代码格式并提高可读性	35
3.10	第 10 题使用 pytest 为简单 Python 函数编写并运行单元测试	36
4	解题感悟	38
4.1	课上练习	38
4.2	课后习题	39
5	上传 GitHub	40

5.1	编写并更新.gitignore	40
5.2	GitHub 仓库链接	41
5.3	commit 提交记录	41

Chapter 1

实验环境说明

1. 系统环境：Ubuntu，用户名 xy_ai，WSL2 架构
2. Shell：bash 5.1.16 (x86_64-pc-linux-gnu)
3. Python：Miniconda 环境中的 Python 3.12；第 12 题使用安装了 PyTorch 的 AI_env 环境进行 CPU 训练
4. 构建工具：build 1.6.1、setuptools 68 及以上版本
5. LaTeX：TeX Live 2022/dev (Debian 版)，使用 XeLaTeX 编译

Chapter 2

课堂学习部分

2.1 第 9 题从源码构建并在干净环境安装 Wheel

第 9 题 从源码构建并在干净环境安装 Wheel

主题: Packaging and Shipping Code 建议用时: 12—15 分钟

在 q09 中按给定内容创建一个最小 Python 命令行包, 并验证 wheel 安装。

给定内容

```
# src/greetlab/cli.py
import argparse
def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print(f"Hello, {a.name}!")

# pyproject.toml
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"
[project]
name = "greetlab-学号"
version = "0.1.0"
[project.scripts]
sdt-greet = "greetlab.cli:main"
```

题目要求

- 创建 src/greetlab/__init__.py、给定的 cli.py 和 pyproject.toml, 并把“学号”替换为自己的学号。
- 在已安装 build 工具的课程环境中运行 python -m build, 生成 wheel。
- 新建一个干净虚拟环境, 只从生成的 wheel 安装。
- 切换到 q09 之外运行 sdt-greet --name <学号>, 确认输出正确。

图 2.1: 第 9 题题目要求

1. 创建项目目录和所需文件

在 assignment-3 下创建 q09, 再建立 src/greetlab 目录, 并创建 __init__.py、

cli.py 和 pyproject.toml。最后用 find 查看目录结构，文件均已放到指定位置。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3$ mkdir q09
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3$ cd q09/
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q09$ mkdir -p src/greetlab
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q09$ touch src/greetlab/__init__.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q09$ nano src/greetlab/cli.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q09$ nano pyproject.toml
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q09$ find .
.
./pyproject.toml
./src
./src/greetlab
./src/greetlab/__init__.py
./src/greetlab/cli.py
```

图 2.2: 创建 q09 项目目录及文件

2. 编写命令行程序

src/greetlab/cli.py 的内容如下：

```
import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print(f"Hello, {a.name}!")
```

程序使用 argparse 接收必填的 --name 参数，然后输出问候语。

3. 编写项目配置文件

pyproject.toml 的内容如下，项目名末尾加上了学号：

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-25020007140"
version = "0.1.0"

[project.scripts]
sdt-greet = "greetlab.cli:main"
```

4. 安装 build 工具并构建 Wheel

第一次执行 `python -m build` 时, 终端提示缺少 build 模块。我先运行 `python -m pip install build` 完成安装, 再重新构建。随后程序创建隔离环境, 读取 `pyproject.toml` 并生成发布文件。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q09$ python -m build
/home/xy_ai/miniconda3/bin/python: No module named build
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q09$ python -m pip install build
Collecting build
  Downloading build-1.6.1-py3-none-any.whl.metadata (5.6 kB)
Requirement already satisfied: packaging>=24.0 in /home/xy_ai/miniconda3/lib/python3.12/site-packages (from build) (24.1)
Collecting pyproject_hooks (from build)
  Downloading pyproject_hooks-1.2.0-py3-none-any.whl.metadata (1.3 kB)
  Downloading build-1.6.1-py3-none-any.whl (31 kB)
  Downloading pyproject_hooks-1.2.0-py3-none-any.whl (10 kB)
Installing collected packages: pyproject_hooks, build
Successfully installed build-1.6.1 pyproject_hooks-1.2.0
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q09$ python -m build
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=68
* Getting build dependencies for sdist...
running egg_info
creating src/greetlab_25020007140.egg-info
writing src/greetlab_25020007140.egg-info/PKG-INFO
writing dependency_links to src/greetlab_25020007140.egg-info/dependency_links.txt
writing entry points to src/greetlab_25020007140.egg-info/entry_points.txt
writing top-level names to src/greetlab_25020007140.egg-info/top_level.txt
writing manifest file 'src/greetlab_25020007140.egg-info/SOURCES.txt'
reading manifest file 'src/greetlab_25020007140.egg-info/SOURCES.txt'
writing manifest file 'src/greetlab_25020007140.egg-info/SOURCES.txt'
* Installed build dependency versions:
```

图 2.3: 安装 build 工具并执行项目构建

构建完成后, `dist` 目录中生成以下两个文件:

```
greetlab_25020007140-0.1.0-py3-none-any.whl
greetlab_25020007140-0.1.0.tar.gz
```

5. 在干净虚拟环境中安装 Wheel

返回 `assignment-3` 目录, 使用下面的命令新建并启用虚拟环境:

```
python -m venv q09-test
source q09-test/bin/activate
pip install q09/dist/greetlab_25020007140-0.1.0-py3-none-any.whl
```

安装命令只指定刚生成的 Wheel。终端显示 `Successfully installed greetlab-25020007140-0.1.0`, 安装顺利完成。

6. 在 q09 目录外验证命令行程序

仍在 `assignment-3` 目录中运行:

```
sdt-greet --name 25020007140
```

终端输出 Hello, 25020007140!。入口命令在 q09 目录外也能运行，因此不是依靠当前源码目录才生效。完成后使用 deactivate 退出虚拟环境。

```
Successfully built greetlab_25020007140-0.1.0.tar.gz and greetlab_25020007140-0.1.0-py3-none-any.whl
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q09$ ls dist
greetlab_25020007140-0.1.0-py3-none-any.whl
greetlab_25020007140-0.1.0.tar.gz
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q09$ cd ..
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3$ python -m venv q09-test
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3$ source q09-test/bin/activate
(q09-test) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3$ pip install
q09/dist/greetlab_25020007140-0.1.0-py3-none-any.whl
Processing ./q09/dist/greetlab_25020007140-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-25020007140
Successfully installed greetlab-25020007140-0.1.0

[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: pip install --upgrade pip
(q09-test) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3$ sdt-greet --
name 25020007140
Hello, 25020007140!
(q09-test) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3$ deactivate
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3$
```

图 2.4: 在干净虚拟环境中安装并验证 Wheel

2.2 第 10 题让编程智能体进入可验证的修复循环

第 10 题 让编程智能体进入可验证的修复循环

主题：智能体编程 建议用时：12—15 分钟

复用第 9 题的 greetlab 包，使用编程智能体完成一个小型测试驱动修复。

题目要求

- 复制 q09 为 q10，添加一个失败测试：当 name 只含空白字符时，main 应以 SystemExit(2) 结束。
- 先运行测试确认失败，再向编程智能体说明目标、约束和测试命令，要求它修改实现并运行测试。
- 人工检查 diff，撤销无关修改，并再次运行测试。

第 7 页

系统开发工具基础 · 每周课上实验检查题

- 在 ai_log.md 中用不超过 5 行记录核心提示、智能体改动和人工验证。

图 2.5: 第 10 题题目要求

1. 复制第 9 题项目并编写测试

在 assignment-3 目录下把 q09 复制为 q10, 再建立 tests/test_cli.py, 用于检查空白用户名的处理结果。

```
cp -r q09 q10
cd q10
mkdir -p tests
nano tests/test_cli.py
```

测试文件如下。这里用 monkeypatch 把 --name 设置为三个空格, 再检查 main() 是否抛出 SystemExit, 并核对退出状态码是否为 2。

```
import sys
import pytest

from greetlab.cli import main

def test_blank_name_exits(monkeypatch):
    monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", "   "])
    with pytest.raises(SystemExit) as exc:
        main()

    assert exc.value.code == 2
```

2. 运行测试并确认失败

在 base 环境的 q10 目录中运行:

```
python -m pytest -q
```

终端显示 1 failed, 报错为 DID NOT RAISE SystemExit。程序仍输出了空白问候语, 说明它把三个空格当成了正常用户名。复现问题后, 我再把修复任务交给编程智能体。

```

(base) xy_ai@Lenovo:~$ cd ~/系统工具开发基础/assignment-3
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3$ cp -r q09 q10
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3$ cd q10
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q10$ mkdir -p tests
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q10$ nano tests/test_cli
.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q10$ python -m pytest -q
F [100%]
===== FAILURES =====
----- test_blank_name_exits -----

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x7eb65aa31760>

    def test_blank_name_exits(monkeypatch):
        monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", "   "])
>       with pytest.raises(SystemExit) as exc:
E         Failed: DID NOT RAISE SystemExit

tests/test_cli.py:9: Failed
----- Captured stdout call -----
Hello,   !
===== short test summary info =====
FAILED tests/test_cli.py::test_blank_name_exits - Failed: DID NOT RAISE Syst
emExit
1 failed in 0.04s
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q10$ |

```

图 2.6: 创建测试并确认空白用户名测试失败

3. 向编程智能体说明目标和约束

在 Codex 中说明当前项目的位置，并给出修复目标、允许修改的文件和验证命令。核心提示如下：

当 `--name` 只包含空白字符时，让 `main` 以 `SystemExit(2)` 结束。只修改 `q10/src/greetlab/cli.py`，不要修改测试，保留正常用户名的输出。请先检查失败原因，修复后运行 `python -m pytest -q`。

智能体读取代码和测试后先运行测试，得到相同的失败结果。原因是 `argparse` 会接收三个空格作为参数值，而原程序解析参数后直接打印，没有继续检查字符串内容。

已修复，只修改了 `q10/src/greetlab/cli.py`，测试文件未改动。

失败原因是参数解析后直接打印问候语，没有检查全空白输入。现在增加：

```
if not a.name.strip():
    raise SystemExit(2)
```

验证结果：

- 修复前：1 failed，提示 DID NOT RAISE SystemExit。
- 修复后运行 `python -m pytest -q`：1 passed。
- 正常用户名输出保持不变，包括名字两端的空格。

图 2.8: 智能体说明修改内容及测试结果

5. 人工检查差异并再次运行测试

回到 `q10` 目录，手动执行下面的命令，对照第 9 题的源码检查修改，然后再次运行测试：

```
diff -u ../q09/src/greetlab/cli.py src/greetlab/cli.py
python -m pytest -q
```

差异中只有空白用户名判断、退出语句和用于分隔代码的空行，没有无关修改。我再次运行测试，终端显示 1 passed in 0.01s。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q10$ diff -u ../q09/src/greetlab/cli.py src/greetlab/cli.py
--- ../q09/src/greetlab/cli.py 2026-09-14 09:45:40.245648492 +0800
+++ src/greetlab/cli.py 2026-09-14 14:22:17.806886562 +0800
@@ -4,4 +4,8 @@
     p = argparse.ArgumentParser()
     p.add_argument("--name", required=True)
     a = p.parse_args()
+
+     if not a.name.strip():
+         raise SystemExit(2)
+
     print(f"Hello, {a.name}!")
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q10$ python -m pytest -q
.
1 passed in 0.01s
[100%]
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q10$ |
```

图 2.9: 人工检查源码差异并再次运行测试

6. 记录核心提示和验证结果

在 `q10/ai_log.md` 中用四行记录本次任务的核心提示、测试命令、智能体改动和人工验证结果，没有超过题目要求的 5 行。

核心提示：空白 `name` 以 `SystemExit(2)` 结束；只改 `cli.py`，不改测试，保留正常输出。

测试命令：在 `q10` 目录运行 `python -m pytest -q`。

智能体改动：解析参数后增加 `strip()` 空白检查；测试由 `1 failed` 变为 `1 passed`。

人工验证：检查 `diff`，无无关修改；再次运行测试，结果为 `1 passed`。

2.3 第 11 题把“无法处理”的协作材料改成可执行信息

第 11 题 把“无法处理”的协作材料改成可执行信息

主题：不止于代码 建议用时：12—15 分钟

围绕“空姓名仍输出问候语”这一缺陷，改写下面三段质量较差的协作材料。

给定内容

```
Issue: Windows 上运行不了，尽快修复。
提交信息: fix bug
评审意见: 这里写得不好，重写。
已知事实: 运行 sdt-greet --name " " 时仍输出 Hello, !，并以 0 退出。
```

题目要求

- 在 `communication.md` 中重写 Issue，包含环境、复现命令、期望结果和实际结果；未知信息写“待确认”。
- 重写提交信息：标题使用祈使语气，正文说明问题和解决方案。
- 重写评审意见：指出具体行为、风险和建议动作，并标注 `Blocking`、`Suggestion` 或 `Nit`。
- 全文控制在 400 字以内，保持专业、简洁。

图 2.10: 第 11 题题目要求

1. 创建 `communication.md`

在 `assignment-3` 下建立 `q11` 目录，进入目录后创建 `communication.md`，用于填写改写后的协作材料。

```
mkdir q11
cd q11
nano communication.md
```

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q10$ cd ..  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3$ mkdir q11  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3$ cd q11  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q11$ nano communication.  
md
```

图 2.11: 创建 q11 目录和 communication.md

2. 改写 Issue

Issue 中写明运行环境为 Windows，给出复现命令 `sdt-greet --name " "`。期望结果是拒绝只含空白字符的 name，并以非零状态退出；实际结果是程序输出空白问候语并以状态码 0 退出。不能确定的问题写为“是否仅在 Windows 下出现，待确认”。

3. 改写提交信息

提交标题使用祈使语气：

```
fix: reject blank name input
```

正文说明处理方法：拒绝只含空白字符的用户名，并使用 `SystemExit(2)` 退出。

4. 改写评审意见

评审中用 `Blocking` 指出程序把无效输入当成了有效输入，并建议通过 `strip()` 检查空白；再用 `Suggestion` 建议补充自动化测试，验证空白输入会触发 `SystemExit(2)`。两条意见分别对应必须修复的问题和后续改进建议。

5. 检查内容和长度

使用 `wc -m communication.md` 统计字符数，结果为 374，没有超过题目规定的 400 字。随后用 `cat communication.md` 查看全文，三部分内容均已写入。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q11$ wc -m communication
.md
374 communication.md
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q11$ cat communication.m
d
# Issue

环境：Windows。

复现命令：sdt-greet --name "  "

期望结果：拒绝仅含空白字符的 name，并以非零状态退出。

实际结果：输出 Hello,  !，并以 0 状态退出。

未知信息：是否仅在 Windows 下出现，待确认。

# Commit Message

fix: reject blank name input

Reject whitespace-only names and exit with SystemExit(2).

# Review

Blocking: 当前实现接受空白 name 并成功退出，会导致无效输入被当作有效输入。建议
使用 strip() 检查，并在为空时退出。

Suggestion: 增加自动化测试，验证空白输入触发 SystemExit(2)。
```

图 2.12: 检查 communication.md 的字符数和内容

2.4 第 12 题修复一个可复现的线性回归训练循环

第 12 题 修复一个可复现的线性回归训练循环

主题: Python 与 PyTorch 建议用时: 12—15 分钟

将下方代码保存为 q12/train.py, 补全 TODO 并完成一次 PyTorch CPU 训练。

给定内容

```
import torch
from torch import nn
torch.manual_seed(20260907)
x = torch.linspace(-1, 1, 101).reshape(-1, 1)
y = 3 * x - 1
model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)
for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    # TODO: 清空梯度、反向传播、更新参数
# TODO: 进入评估模式并在 no_grad 中打印最终损失、weight 和 bias
```

题目要求

- 补全训练循环, 正确调用 zero_grad、backward 和 step。
- 训练后调用 model.eval(), 并在 torch.no_grad() 中计算最终损失。

第 8 页

系统开发工具基础·每周课上实验检查题

- 打印最终损失、weight 和 bias; 最终损失应小于 0.001。
- 不得直接把 weight 和 bias 赋值为 3 和 -1。

图 2.13: 第 12 题题目要求

1. 激活环境并创建训练文件

本题需要使用 PyTorch, 因此激活已经安装 PyTorch 的 AI_env 环境, 进入 assignment-3 下的 q12 目录并创建 train.py。

```
conda activate AI_env
cd ~/系统工具开发基础/assignment-3/q12
nano train.py
```

2. 补全训练循环

根据题目给出的代码, 在每轮训练中依次计算预测值和损失, 再调用 zero_grad()、backward() 和 step():

```

for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)

    opt.zero_grad()
    loss.backward()
    opt.step()

```

3. 计算并输出最终结果

训练结束后调用 `model.eval()`，并在 `torch.no_grad()` 中计算最终损失。最后打印 final loss、weight 和 bias。代码没有把张量和模型移到 CUDA，所以本次训练使用 CPU。

```

(AI_env) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q12$ cat train.py
import torch
from torch import nn

torch.manual_seed(20260907)

x = torch.linspace(-1, 1, 101).reshape(-1, 1)
y = 3 * x - 1

model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)

for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)

    opt.zero_grad()
    loss.backward()
    opt.step()

model.eval()

with torch.no_grad():
    final_pred = model(x)
    final_loss = loss_fn(final_pred, y)

print("final loss:", final_loss.item())
print("weight:", model.weight.item())
print("bias:", model.bias.item())
(AI_env) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q12$

```

图 2.14: 补全后的 train.py 代码

4. 运行程序并检查结果

连续两次执行 `python train.py`，得到相同结果：

- final loss: 2.3923129102709773e-12, 小于 0.001;
- weight: 2.99999737739563, 接近 3;

- bias: -0.9999998807907104, 接近 -1。

```
(AI_env) xy_ai@Lenovo:~$ cd ~/系统工具开发基础/assignment-3/q12
(AI_env) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q12$ nano train.py
(AI_env) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q12$ python train.py
final loss: 2.3923129102709773e-12
weight: 2.99999737739563
bias: -0.9999998807907104
(AI_env) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q12$ python train.py
final loss: 2.3923129102709773e-12
weight: 2.99999737739563
bias: -0.9999998807907104
(AI_env) xy_ai@Lenovo:~/系统工具开发基础/assignment-3/q12$ |
```

图 2.15: 两次运行线性回归训练程序的结果

Chapter 3

课后习题

3.1 第 1 题创建并安装带有 pyproject.toml 的 Python 包

1. 创建项目配置与包目录

在 assignment-3/homework/01 目录中创建 pyproject.toml，使用 setuptools 作为构建后端，并填写软件包名称、版本、说明和 Python 版本要求：

```
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "hello-package"
version = "0.1.0"
description = "A simple Python package"
requires-python = ">=3.10"
```

接着创建 hello_package 目录和 __init__.py，得到一个最小的 Python 包结构。

```
mkdir hello_package
touch hello_package/__init__.py
```

2. 创建虚拟环境并安装软件包

使用 Python 自带的 venv 模块创建并激活虚拟环境。which python 返回当前项目下的 .venv/bin/python，后面的安装操作都在这个环境中完成。

```
python3 -m venv .venv
source .venv/bin/activate
which python
pip install .
pip list
```

pip 根据 pyproject.toml 构建 Wheel 并完成安装。随后查看软件包列表，其中 hello-package 的版本为 0.1.0。

操作失误，第三次实验作业不小心放到第四次里面了，现在已经移动到第三次实验了。

```
(base) xy_ai@Lenovo:~$ cd ~/系统工具开发基础/assignment-4/homework/01
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01$ nano pyproject.toml
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01$ cat pyproject.toml
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "hello-package"
version = "0.1.0"
description = "A simple Python package"
requires-python = ">=3.10"
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01$ mkdir hello_package
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01$ touch hello_package/__init__.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01$ ls
hello_package  pyproject.toml
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01$ python3 -m venv .venv
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01$ source .venv/bin/activate
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01$ which python
/home/xy_ai/系统工具开发基础/assignment-4/homework/01/.venv/bin/python
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01$ pip install .
Processing /home/xy_ai/系统工具开发基础/assignment-4/homework/01
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: hello-package
  Building wheel for hello-package (pyproject.toml) ... done
  Created wheel for hello-package: filename=hello_package-0.1.0-py3-none-any.whl size=1220 sha256
  f7542b
  Stored in directory: /tmp/pip-ephem-wheel-cache-z6wcx81g/wheels/91/4b/b8/d47e358b1925239f2202d
  Successfully built hello-package
Installing collected packages: hello-package
Successfully installed hello-package-0.1.0
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01$ pip list
Package      Version
-----
hello-package 0.1.0
pip           24.0
```

图 3.1: 创建 Python 包并在虚拟环境中安装

3. 创建并检查锁文件

在虚拟环境中安装 uv，查看版本后运行 uv lock，生成 uv.lock：

```
pip install uv
uv --version
uv lock
cat uv.lock
head -n 40 uv.lock
```

本次使用的 uv 版本为 0.12.15。锁文件开头写有格式版本、修订号和 requires-python = >=3.10；软件包条目记录了 hello-package、版本 0.1.0 和本地可编辑来源，与 pyproject.toml 中的配置一致。

操作失误，第三次实验作业不小心放到第四次里面了，现在已经移动到第三次实验了。

```
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01$ pip install uv
Collecting uv
  Downloading uv-0.12.15-py3-none-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (11 kB)
  Downloading uv-0.12.15-py3-none-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (20.1 MB)
    20.1/20.1 MB 5.2 MB/s eta 0:00:00
Installing collected packages: uv
Successfully installed uv-0.12.15
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01$ uv --version
uv 0.12.15 (x86_64-unknown-linux-gnu)
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01$ uv lock
Resolved 1 package in 2ms
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01$ ls
build hello_package hello_package.egg-info pyproject.toml uv.lock
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01$ cat uv.lock
version = 1
revision = 3
requires-python = ">=3.10"

[[package]]
name = "hello-package"
version = "0.1.0"
source = { editable = "." }
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01$ head -n 40 uv.lock
version = 1
revision = 3
requires-python = ">=3.10"

[[package]]
name = "hello-package"
version = "0.1.0"
source = { editable = "." }
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/01$ ls -la
total 32
drwxr-xr-x 6 xy_ai xy_ai 4096 Sep 15 22:18 .
drwxr-xr-x 3 xy_ai xy_ai 4096 Sep 15 22:13 ..
drwxr-xr-x 5 xy_ai xy_ai 4096 Sep 15 22:16 .venv
drwxr-xr-x 4 xy_ai xy_ai 4096 Sep 15 22:17 build
drwxr-xr-x 2 xy_ai xy_ai 4096 Sep 15 22:15 hello_package
drwxr-xr-x 2 xy_ai xy_ai 4096 Sep 15 22:17 hello_package.egg-info
-rw-r--r-- 1 xy_ai xy_ai 213 Sep 15 22:15 pyproject.toml
-rw-r--r-- 1 xy_ai xy_ai 134 Sep 15 22:18 uv.lock
```

图 3.2: 使用 uv 生成并检查锁文件

3.2 第 2 题使用现有的 pyproject.toml 项目执行构建并检查发布文件

1. 创建并配置待构建项目

在 assignment-3/homework 下创建 02 目录，再建立 hello_build 包目录和 __init__.py。现有的 pyproject.toml 内容如下：

```
[build-system]
requires = ["setuptools"]
build-backend = "setuptools.build_meta"

[project]
name = "hello-build"
version = "0.1.0"
description = "A simple build example"
requires-python = ">=3.10"
```

```
mkdir 02
cd 02
mkdir hello_build
touch hello_build/__init__.py
```

2. 执行构建并处理依赖获取失败

第一次构建时，隔离环境需要从 PyPI 获取 setuptools 和 wheel，但当时网络连接被拒绝，终端提示 Failed to build。调整 pyproject.toml 后，我加上 --no-isolation，直接使用当前 Python 环境中已安装的构建依赖：

```
python3 -m build --no-isolation
```

这次构建顺利完成，同时生成了源码分发和 Wheel。

操作失误，第三次实验作业不小心放到第四次里面了，现在已经移动到第三次实验了。

```

(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ mkdir 02
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ cd 02
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ mkdir hello_build
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ touch hello_build/__init__.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ nano pyproject.toml
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ ~ /系统工具开发基础/assignment-4/homework/02$
Building source distribution...
error: Failed to build
      '/home/xy_ai/系统工具开发基础/assignment-4/homework/02'
  cause: Failed to resolve requirements from 'build-system.requires'
  cause: No solution found when resolving: 'setuptools>=68', 'wheel'
  cause: Request failed after 3 retries in 4.9s
  cause: Failed to fetch: 'https://pypi.org/simple/wheel/'
  cause: error sending request for url (https://pypi.org/simple/wheel/)
  cause: client error (Connect)
  cause: tunnel error: failed to create underlying connection
  cause: tcp connect error
  cause: Connection refused (os error 111)
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ nano pyproject.toml
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ python3 -m build --no-isolation
* Getting build dependencies for sdist...
running egg_info
creating hello_build.egg-info
writing hello_build.egg-info/PKG-INFO
writing dependency_links to hello_build.egg-info/dependency_links.txt
writing top-level names to hello_build.egg-info/top_level.txt

```

图 3.3: 创建项目并处理隔离构建的依赖获取失败

3. 确认构建工具并重复构建

`python3 -m pip show build` 显示 `build` 的版本为 1.6.1。随后再次执行无隔离构建，两个发布文件仍能正常生成。

操作失误，第三次实验作业不小心放到第四次里面了，现在已经移动到第三次实验了。

```

Successfully built hello_build-0.1.0.tar.gz and hello_build-0.1.0-py3-none-any.whl
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ python3 -m pip show build
Name: build
Version: 1.6.1
Summary: A simple, correct Python build frontend
Home-page: https://build.pypa.io
Author:
Author-email: Filipe Lains <lains@riseup.net>, Bernát Gábor <gaborjbernát@gmail.com>, layday <laydayil.com>
License:
Location: /home/xy_ai/miniconda3/lib/python3.12/site-packages
Requires: packaging, pyproject_hooks
Required-by:
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ python3 -m build --no-isolation
* Getting build dependencies for sdist...
running egg_info
writing hello_build.egg-info/PKG-INFO
writing dependency_links to hello_build.egg-info/dependency_links.txt
writing top-level names to hello_build.egg-info/top_level.txt
reading manifest file 'hello_build.egg-info/SOURCES.txt'
writing manifest file 'hello_build.egg-info/SOURCES.txt'
* Building sdist...
running sdist
running egg_info
writing hello_build.egg-info/PKG-INFO
writing dependency_links to hello_build.egg-info/dependency_links.txt
writing top-level names to hello_build.egg-info/top_level.txt

```

图 3.4: 检查 `build` 版本并再次成功构建

4. 检查生成的发布文件

使用 `ls dist` 查看发布目录，其中包含两个文件：

```

hello_build-0.1.0-py3-none-any.whl
hello_build-0.1.0.tar.gz

```

再运行 `file dist/*.whl`, 终端把 Wheel 识别为使用 deflate 压缩的 Zip archive data。也就是说, Wheel 文件可以按 ZIP 归档查看。

操作失误, 第三次实验作业不小心放到第四次里面了, 现在已经移动到第三次实验了。

```
adding 'hello_build-0.1.0.dist-info/top_level.txt'
adding 'hello_build-0.1.0.dist-info/RECORD'
removing build/bdist.linux-x86_64/wheel
Successfully built hello_build-0.1.0.tar.gz and hello_build-0.1.0-py3-none-any.whl
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ ls dist
hello_build-0.1.0-py3-none-any.whl  hello_build-0.1.0.tar.gz
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ file dist/*.whl
dist/hello_build-0.1.0-py3-none-any.whl: Zip archive data, at least v2.0 to extract, compression method=deflate
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ |
```

图 3.5: 检查 dist 目录及 Wheel 文件类型

3.3 第 3 题安装第二题构建的 Wheel 包并检查安装结果

1. 创建并激活虚拟环境

在 assignment-3/homework 下创建 03 目录，并在其中建立独立的 Python 虚拟环境，避免这次安装影响其他环境。

```
mkdir -p 03
cd 03
python3 -m venv .venv
source .venv/bin/activate
```

2. 安装第二题生成的 Wheel

使用相对路径指定第二题 dist 目录中的 Wheel 文件并执行安装：

```
pip install ../02/dist/hello_build-0.1.0-py3-none-any.whl
```

pip 读取本地 Wheel 后安装 hello-build，终端显示 Successfully installed hello-build-0.1.0。

3. 使用 pip show 检查安装结果

pip list 中已经出现版本为 0.1.0 的 hello-build。继续运行 pip show hello-build，名称、版本和说明都与第二题的 pyproject.toml 一致，安装位置在当前虚拟环境的 site-packages 目录。

操作失误，第三次实验作业不小心放到第四次里面了，现在已经移动到第三次实验了。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/02$ cd ~/系统工具开发基础/assignment-4/homework
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ mkdir -p 03
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ cd 03
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$ python3 -m venv .venv
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$ source .venv/bin/activate
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$ pip install ../02/dist/hello_build-0.1.0-py3-none-any.whl
Processing /home/xy_ai/system_tool_dev_foundation/assignment-4/homework/02/dist/hello_build-0.1.0-py3-none-any.whl
Installing collected packages: hello-build
Successfully installed hello-build-0.1.0
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$ pip list
Package Version
-----
hello-build 0.1.0
pip 24.0
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$ pip show hello-build
Name: hello-build
Version: 0.1.0
Summary: A simple build example
Home-page:
Author:
Author-email:
License:
Location: /home/xy_ai/system_tool_dev_foundation/assignment-4/homework/03/.venv/lib/python3.12/site-packages
Requires:
Required-by:
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$ |
```

图 3.6: 安装第二题构建的 Wheel 并检查安装结果

3.4 第 4 题查看 Python 包的依赖信息和安装位置

1. 查看软件包元数据与依赖信息

激活第 3 题创建的虚拟环境，再使用 `pip show` 查看已安装软件包的详细信息：

```
source .venv/bin/activate
pip show hello-build
```

输出中 Name 为 `hello-build`，Version 为 `0.1.0`，Summary 为 `A simple build example`。Requires 为空，表示该包没有声明运行依赖；Required-by 为空，表示当前环境中没有其他包依赖它。

2. 查看安装位置

`pip show` 的 Location 字段显示软件包安装在当前虚拟环境的 `.venv/lib/python3.12/site-packages` 目录。随后导入 `hello_build` 并输出模块的 `__file__` 属性，以确认实际加载的文件位置：

```
python -c "import hello_build; print(hello_build.__file__)"
```

终端返回当前虚拟环境中 `hello_build/__init__.py` 的完整路径，与 `pip show` 给出的安装目录相符。

操作失误，第三次实验作业不小心放到第四次里面了，现在已经移动到第三次实验了。

```
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$ source .venv/bin/activate
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$ pip show hello-build
Name: hello-build
Version: 0.1.0
Summary: A simple build example
Home-page:
Author:
Author-email:
License:
Location: /home/xy_ai/系统工具开发基础/assignment-4/homework/03/.venv/lib/python3.12/site-packages
Requires:
Required-by:
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$ pip list
Package      Version
-----
hello-build  0.1.0
pip          24.0
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$ python -c "import hello_build; print(hello_build.__file__)"
/home/xy_ai/系统工具开发基础/assignment-4/homework/03/.venv/lib/python3.12/site-packages/hello_build/__init__.py
(.venv) (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/03$ |
```

图 3.7: 查看 `hello-build` 的依赖信息和实际安装位置

3.5 第 5 题使用 Claude 智能体读取代码并自动生成说明文件

1. 创建并运行示例程序

在 assignment-3/homework 下创建 05 目录，并编写 calculator.py。其中 add(a, b) 负责加法，multiply(a, b) 负责乘法。运行程序后，终端依次输出 5 和 20。

```
mkdir -p 05
cd 05
nano calculator.py
python3 calculator.py
```

2. 启动 Claude 并说明任务要求

在项目目录运行 claude，进入 Claude Code 交互界面，本次版本为 2.1.236。我要求智能体读取 calculator.py，说明函数和运行结果，不改源代码，只在当前目录新建 README.md。

操作失误，第三次实验作业不小心放到第四次里面了，现在已经移动到第三次实验了。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ mkdir -p 05
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ cd 05
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/05$ nano calculator.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/05$ python3 calculator.py
5
20
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/05$ claude
```

Welcome back!



deepseek-v4-pro · API Usage Billing
\\wsl.localhost\Ubuntu\home\xy_ai\系统工具开发基...

Tips for getting started
Run /init to creat...

What's new
Added the 'SendFee...
Added '{id, text, ...
Added a tip on Bas...
/release-notes for...

请作为编程智能体完成这个任务：

1. 阅读当前目录中的 calculator.py。
2. 分析程序中有哪些函数，以及程序运行后会输出什么。
3. 不要修改 calculator.py。
4. 在当前目录中创建 README.md。
5. 在 README.md 中简洁说明程序功能、两个函数分别做什么，以及程序的运行结果。

图 3.8: 运行示例程序并向 Claude 智能体提交任务

3. 由智能体分析代码并生成 README

Claude 读取 `calculator.py` 后找到了 `add` 和 `multiply` 两个函数，并写出一份 21 行的 `README.md`。文件分为程序功能、函数说明和运行结果三部分，其中也写明了示例输出 5 和 20。

写完后，智能体重新读取 `README.md` 并汇总结果。`calculator.py` 没有发生改动，符合本题要求。

操作失误，第三次实验作业不小心放到第四次里面了，现在已经移动到第三次实验了。

```
● 我已读取 calculator.py, 程序包含两个函数 add 和 multiply。现在创建 README.md

● Write(README.md)
  | Wrote 21 lines to README.md
    1 # calculator.py 说明
    2
    3 ## 程序功能
    5 这是一个简单的计算器程序，实现了加法和乘法两个基本运算功能，并输出两个示例运算的结果。
    6
    7 ## 函数说明
    9 - 'add(a, b)': 加法函数，接收两个参数，返回 'a + b' 的值。
   10 - 'multiply(a, b)': 乘法函数，接收两个参数，返回 'a * b' 的值。
    ... +13 lines (ctrl+o to expand)

Thought for 2s, read 1 file (ctrl+o to expand)

● 按任务要求，重新读取 README.md 检查内容是否正确

Read 1 file (ctrl+o to expand)

● 任务已完成。总结如下：

1. 分析 calculator.py

程序包含两个函数：
- add(a, b): 返回两数之和
- multiply(a, b): 返回两数之积

程序运行后输出：
5
20
即 add(2, 3) = 5, multiply(4, 5) = 20。

2. 创建的 README.md - 已写入 \\wsl.localhost\Ubuntu\home\xy_ai\系统工具开发基础\assignment-4\homework\05\README.md, 包含三部分内容：
- 程序功能：实现加法与乘法两种运算的简单计算器
- 函数说明：分别说明 add 和 multiply 的作用
- 运行结果：展示输出 5 和 20

3. 检查验证 - 已重新读取 README.md, 内容与 calculator.py 的实际逻辑一致，无错误。calculator.py 未被修改。

* Baked for 19s
```

图 3.9: Claude 分析代码并生成 README.md

4. 退出智能体并检查生成文件

输入 `/exit` 退出 Claude Code, 再运行 `ls`。目录中保留了 `calculator.py`, 同时多出了新生成的 `README.md`。

操作失误, 第三次实验作业不小心放到第四次里面了, 现在已经移动到第三次实验了。

```
> /exit

/exit          Exit the CLI
/context        Visualize current context usage as a
                 colored grid
/doctor         Health-check the user's Claude Code setup
                 and fix issues: diagnose installation hea...
/verify         Verify that a code change actually does
                 what it's supposed to by exercising it en...
/agents         (removed) Ask Claude to create/manage
                 subagents, or edit .claude/agents/
/memory         Edit CLAUDE.md files and memory settings
/autocompact    Set how full the context gets before
                 auto-summarizing
/subtask        Send a subagent off with your full
                 context; its result comes back here
/clear          Start a new session with empty context;
                 previous session stays on disk (resumable...
/compact        Free up context by summarizing the
                 conversation so far

Resume this session with:
claude --resume 80bc9ed1-3a1a-458a-bb8f-d54e2ac87e84
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/05$ ls
README.md  calculator.py
```

图 3.10: 退出 Claude Code 并查看生成文件

最后用 `cat README.md` 查看文件。内容包括加法、乘法两个函数的作用、参数和返回值, 也列出了程序运行时的输出 5 和 20。

操作失误, 第三次实验作业不小心放到第四次里面了, 现在已经移动到第三次实验了。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/05$ cat README.md
# calculator.py 说明

## 程序功能

这是一个简单的计算器程序，实现了加法和乘法两个基本运算功能，并输出两个示例运算的结果。

## 函数说明

- `add(a, b)`：加法函数，接收两个参数，返回 `a + b` 的值。
- `multiply(a, b)`：乘法函数，接收两个参数，返回 `a * b` 的值。

## 运行结果

程序依次调用两个函数并打印结果：

```
5
20
```
```

图 3.11: 检查 Claude 自动生成的 README.md

3.6 第 6 题让 Claude 智能体发现并修复 Python 程序中的错误

1. 创建错误程序并复现问题

在 assignment-3/homework 下创建 06 目录，并编写 divide.py。divide(a, b) 直接返回 a / b ，程序分别用 2 和 0 作除数调用该函数。

```
def divide(a, b):  
    return a / b  
  
print(divide(10, 2))  
print(divide(10, 0))
```

第一个调用输出 5.0，第二个调用则触发 ZeroDivisionError: division by zero，程序在这里中断。

操作失误，第三次实验作业不小心放到第四次里面了，现在已经移动到第三次实验了。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/05$ cd ~/系统工具开发基础/assignment-4/homework  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ mkdir -p 06  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ cd 06  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/06$ cd 06  
-bash: cd: 06: No such file or directory  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/06$ nano divide.py  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/06$ python3 divide.py  
5.0  
Traceback (most recent call last):  
  File "/home/xy_ai/系统工具开发基础/assignment-4/homework/06/divide.py", line 6, in <module>  
    print(divide(10, 0))  
    ~~~~~  
  File "/home/xy_ai/系统工具开发基础/assignment-4/homework/06/divide.py", line 2, in divide  
    return a / b  
    ~~~~  
ZeroDivisionError: division by zero  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/06$ claud
```

图 3.12: 运行 divide.py 并复现除零错误

2. 向 Claude 提交自动修复任务

在当前目录启动 Claude Code。我要求智能体读取并运行 divide.py，找到报错原因后直接修改文件；修改完成还要再次运行，直到程序能够正常结束。

Claude 很快定位到 `divide(10, 0)`。不过第一次运行时，Windows 环境中找不到 `python3`；改用 WSL 后又遇到路径转换问题。调整命令后，程序终于在 Ubuntu 中运行。

操作失误，第三次实验作业不小心放到第四次里面了，现在已经移动到第三次实验了。

```
请作为编程智能体完成以下任务：
1. 阅读当前目录中的 divide.py。
2. 运行程序，找出程序报错的原因。
3. 直接修改 divide.py，使除数为 0 时程序不会崩溃。
4. 修改后再次运行程序验证。
5. 如果仍有错误，继续修改，直到程序能够正常运行。
6. 不要只告诉我怎么改，请直接读取、修改并运行文件完成任务。

Thought for 5s, read 1 file (ctrl+o to expand)
• 文件内容很简单：divide(a, b) 直接做 a / b，而第 6 行调用了 divide(10, 0)。现在运行确认报错：
• Bash(python3 "\\wsl.localhost\Ubuntu\home\xy_ai\系统工具开发基础\assignment-4\homework\06\divide.py")
  | Error: Exit code 49
  | Python was not found; run without arguments to install from the Microsoft Store, or disable this shortcut from Settings > Apps > Advanced app settings > App execution aliases.

Thought for 2s (ctrl+o to expand)
• Windows 上没有 python3 别名。这是 WSL 路径，通过 wsl 在 Ubuntu 内运行
• Bash(wsl python3 "/home/xy_ai/系统工具开发基础/assignment-4/homework/06/divide.py")
  | Error: Exit code 2
  | python3: can't open file
  | '/home/xy_ai/系统工具开发基础/assignment-4/homework/06/D:/Git/Git/home/xy_ai/系统工具开发基础/assignment-4/homework/06/divide.py':
  | [Errno 2] No such file or directory

Thought for 3s (ctrl+o to expand)
```

图 3.13: Claude 分析除零错误并处理运行环境问题

3. 由智能体修改代码并验证

Claude 在 `divide` 函数中增加了除数判断。`b == 0` 时返回提示字符串，其他情况仍执行原来的除法：

```
def divide(a, b):
    if b == 0:
        return "Error: division by zero!"
    return a / b
```

修改后重新运行，终端依次输出 `5.0` 和 `Error: division by zero!`，没有再出现异常回溯。

操作失误，第三次实验作业不小心放到第四次里面了，现在已经移动到第三次实验了。

```
● Update(divide.py)
└─ Added 2 lines
   1 def divide(a, b):
   2 +     if b == 0:
   3 +         return "Error: division by zero!"
   4     return a / b
   5
   6

Thought for 2s (ctrl+o to expand)

● 修改完成：除数 b 为 0 时返回提示信息而不是直接报错。再次运行验证

● Bash(MSYS_NO_PATHCONV=1 wsl -e python3 "/home/xy_ai/系统工具开发基础/assignment-4/homework/06/divide.py")
└─ 5.0
   Error: division by zero!

Thought for 2s (ctrl+o to expand)

● 任务完成。总结如下：

完成情况

1. 读取 divide.py：发现 divide(a, b) 直接执行 a / b，且第 6 行调用了 divide(10, 0)。

2. 运行并定位报错：程序在第 6 行触发 ZeroDivisionError: division by zero 崩溃。

3. 修改代码：在 divide 函数中加入除数是否为 0 的判断：

def divide(a, b):
    if b == 0:
        return "Error: division by zero!"
    return a / b

4. 再次运行验证：程序正常结束，输出：

5.0
Error: division by zero!
```

图 3.14: Claude 修改 divide.py 并运行验证

4. 检查最终代码与运行结果

退出智能体后，我用 `cat divide.py` 查看最终代码，再手动执行 `python3 divide.py`。两次输出与智能体的运行结果一致，除零时程序不会再崩溃。

操作失误，第三次实验作业不小心放到第四次里面了，现在已经移动到第三次实验了。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/06$ cat divide.py
def divide(a, b):
    if b == 0:
        return "Error: division by zero!"
    return a / b

print(divide(10, 2))
print(divide(10, 0))
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/06$ python3 divide.py
5.0
Error: division by zero!
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/06$
```

图 3.15: 检查修复后的代码并再次运行程序

3.7 第 7 题编写一个最简单的 README

1. 创建简单 Python 项目

在 assignment-3/homework 下创建 07 目录，并在其中建立 hello.py。该程序只包含一条输出语句：

```
print("Hello, World!")
```

2. 编写 README.md

在项目目录中创建 README.md，写清项目用途、文件组成、运行命令和输出结果：

```
# Hello Project

这是一个简单的 Python 程序，用于输出 `Hello, World!`。

## 文件说明
- `hello.py`: Python 程序文件。
- `README.md`: 项目说明文件。

## 运行方法
python3 hello.py

## 运行结果
Hello, World!
```

按照 README 中的命令即可运行程序，不需要再打开源码查找入口。

操作失误，第三次实验作业不小心放到第四次里面了，现在已经移动到第三次实验了。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ mkdir -p 07
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ cd 07
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/07$ nano hello.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/07$ nano README.md
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/07$ nano README.md
```

图 3.16: 创建 hello.py 并编写 README.md

3.8 第 8 题编写一个规范的 Bug 报告

1. 创建并运行错误程序

在 assignment-3/homework 下创建 08 目录，并编写 bug.py。程序建立一个只有三个元素的列表，却尝试访问下标为 5 的元素：

```
numbers = [1, 2, 3]
print(numbers[5])
```

运行 python3 bug.py 后，程序在第 2 行触发 IndexError: list index out of range。终端同时给出了文件路径和出错代码行，这些信息都要写进 Bug 报告。

2. 编写 BUG_REPORT.md

创建 BUG_REPORT.md，按照题目要求依次记录环境、期望结果、实际结果和复现步骤：

```
# Bug Report

## 环境
- 系统: WSL2 Ubuntu
- Python: Python 3

## 期望结果
程序正常输出列表中的数字。

## 实际结果
程序出现 IndexError。

## 复现步骤
1. 进入程序目录。
2. 运行 `python3 bug.py`。
3. 程序出现 IndexError。
```

复现步骤只保留进入目录和运行程序这两个必要操作。期望结果与实际结果分开写，查看报告时可以直接看出问题所在。

操作失误，第三次实验作业不小心放到第四次里面了，现在已经移动到第三次实验了。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/07$ cd ~/系统工具开发基础/assignment-4/homework
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ mkdir -p 08
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ cd 08
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/08$ nano bug.py
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/08$ python3 bug.py
Traceback (most recent call last):
  File "/home/xy_ai/系统工具开发基础/assignment-4/homework/08/bug.py",
    line 2, in <module>
      print(numbers[5])
            ~~~~~~^^^^
IndexError: list index out of range
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/08$ nano BUG_REPORT.md
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/08$
```

图 3.17: 复现 IndexError 并创建规范的 Bug 报告

3.9 第 9 题手动规范 Python 代码格式并提高可读性

1. 创建并查看待整理代码

在 assignment-3/homework 下创建 09 目录，并编写 messy.py。原代码能够完成加法运算，但运算符和函数参数之间缺少空格：

```
def add(a, b):  
    return a+b  
  
print(add(2,3))
```

代码虽然能运行，但空格使用不统一，看起来比较挤。

2. 手动规范代码格式

重新编辑 messy.py，在加法运算符两侧和函数调用的逗号后增加空格，并在函数定义与顶层调用之间保留两个空行。整理后的代码如下：

```
def add(a, b):  
    return a + b  
  
print(add(2, 3))
```

这次只改格式，没有动 add 的计算逻辑。执行 python3 messy.py 后仍输出 5。

操作失误，第三次实验作业不小心放到第四次里面了，现在已经移动到第三次实验了。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ mkdir -p  
09  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ cd 09  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/09$ nano  
messy.py  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/09$ cat m  
essy.py  
def add(a,b):  
    return a+b  
  
print(add(2,3))  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/09$ nano  
messy.py  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/09$ pytho  
n3 messy.py  
5
```

图 3.18: 手动规范 messy.py 的代码格式并运行

3.10 第 10 题使用 pytest 为简单 Python 函数编写并运行单元测试

1. 编写待测试函数

在 assignment-3/homework 下创建 10 目录，并在 calculator.py 中编写加法函数：

```
def add(a, b):  
    return a + b
```

2. 编写单元测试

创建 test_calculator.py，从 calculator 模块导入 add，再通过断言检查 add(2, 3) 的返回值是否等于 5：

```
from calculator import add  
  
def test_add():  
    assert add(2, 3) == 5
```

测试文件和测试函数都以 test_ 开头，Pytest 可以自动找到它。

3. 运行测试并查看结果

在项目目录执行安静模式测试：

```
pytest -q
```

终端显示 1 passed in 0.01s。运行 ls 后可以看到 calculator.py、test_calculator.py，以及测试时生成的 __pycache__ 目录。

操作失误，第三次实验作业不小心放到第四次里面了，现在已经移动到第三次实验了。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ mkdir -p 10  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework$ cd 10  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/10$ nano calculator.py  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/10$ nano test_calculator.py  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/10$ pytest -q  
.  
1 passed in 0.01s [100%]  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/10$ ls  
__pycache__ calculator.py test_calculator.py  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-4/homework/10$
```

图 3.19: 使用 Pytest 运行加法函数的单元测试

Chapter 4

解题感悟

4.1 课上练习

1. 第 9 题是我第一次把一个小项目从源码构建成 Wheel，再放到新虚拟环境中安装。特意离开源码目录运行 `sdt-greet` 后仍能得到正确输出，这一步比只看“安装成功”更直观。
2. 第 10 题里最有用的是先保留失败测试。它把“空白用户名处理不对”变成了明确的 `1 failed`。智能体修好后，我又看了一遍差异并重跑测试，能够确定它没有顺手改动别的文件。
3. 第 11 题让我发现，“程序无法处理”这种说法几乎不能帮助别人定位问题。把环境、命令、期望和实际结果补齐后，同一个问题就清楚多了；评审意见也应分清必须修改和建议改进。
4. 第 12 题把 PyTorch 训练循环的顺序串了起来：先清空梯度，再反向传播，最后更新参数。训练结束不能只看程序是否跑完，还要检查 `loss`、`weight` 和 `bias` 是否接近题目给出的目标。

4.2 课后习题

1. 第 1 题实际做下来，我才把“写配置、建虚拟环境、安装本地包、生成锁文件”这几步连在一起。以前看到 `pyproject.toml` 和锁文件时只知道它们与依赖有关，这次通过 `pip list` 和 `uv.lock` 对照，能看懂包名、版本和来源记录在哪里了。
2. 第 6 题给我的印象最深。除零错误本身不难，但 Claude 一开始受 Windows 和 WSL 路径影响，连续两次没有把程序跑起来。这让我意识到，智能体给出修改并不等于任务已经完成，还要看它是否在正确的环境中真正运行过。最后我又手动执行了一次，心里才有底。
3. 第 10 题让我体会到单元测试的作用。以前修改小函数时通常只看一次输出，现在可以把 `add(2, 3) == 5` 写成断言，以后代码发生变化时直接运行 Pytest，很快就能知道原来的功能有没有被改坏。

Chapter 5

上传 GitHub

5.1 编写并更新.gitignore

提交第三次实验前，我在课程仓库根目录继续更新 .gitignore。本次实验会产生虚拟环境、Python 构建目录、egg-info 元数据和测试缓存，如果全部上传，不但文件数量多，也会把本机环境带进仓库。因此，在原有规则后补充了第三次实验相关内容：

```
# 第三次实验的虚拟环境和 Python 构建中间产物
/assignment-3/q09-test/
/assignment-3/**/build/
/assignment-3/**/*egg-info/

# q10 从 q09 复制的旧构建包；q09/dist 保留为第 9 题实验成果
/assignment-3/q10/dist/

# 第三次实验报告的排版检查图片和本地备份
/assignment-3/实验报告/tmp/
/assignment-3/实验报告/*.bak
.venv/
```

其中，q09-test 和 .venv 都是可以重新创建的虚拟环境；build、egg-info 和 q10/dist 属于构建过程中产生的文件。实验报告的临时检查图片和备份文件也没有提交。这样处理后，仓库中保留源码、测试、作业截图和正式报告，提交内容更清楚。

5.2 GitHub 仓库链接

本课程作业保存在下面的公开仓库中，默认分支为 `main`：

https://github.com/xiaoyang1088/system_development_tools

仓库用户名为 `xiaoyang1088`，仓库名为 `system_development_tools`。第三次实验位于 `assignment-3/` 目录，其中包括课堂练习 `q09-q12`、课后练习 `homework/`、实验截图和实验报告。

5.3 commit 提交记录

第三次实验分三次提交。2026 年 9 月 14 日先提交了针对第三次实验补充的 `.gitignore`，对应提交号 `246d4c9`；9 月 15 日提交课堂练习 `q09-q12`，对应提交号 `2d3b9b7`；随后提交课后练习 `homework`，对应提交号 `b861116`。分开提交后，每次修改的范围比较明确，后续查看或回退时也更方便。

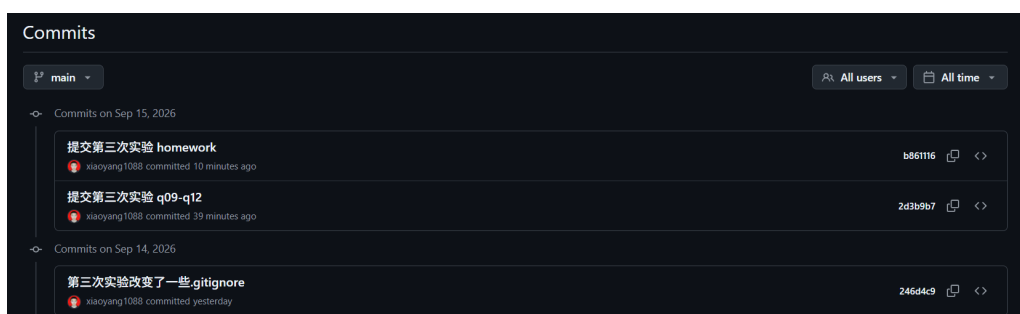


图 5.1: 第三次实验作业的 GitHub commit 提交记录